MONITORAMENTO DE MODELOS DE ML

Ciclo de Vida, Drift, PSI e MLOps em Producao -- Nivel Especialista

Breno Ferrari · Preparacao Claro · Abril 2026

#	Secao
1	Por que Modelos Degradam em Producao
2	Taxonomia do Ciclo de Vida de Modelos
3	Data Drift -- Conceito, Tipos e Deteccao
4	PSI -- Population Stability Index
5	Concept Drift -- Quando o Mundo Muda
6	Monitoramento de Performance
7	Monitoramento de Features e Dados
8	Monitoramento de Predictions (Output Drift)
9	SHAP em Monitoramento
10	Alertas, SLAs e Estrategias de Resposta
11	Retreinamento -- Quando e Como
12	MLOps -- Ferramentas e Arquitetura
13	Pipeline Completo de Monitoramento em Producao
14	Checklist do Especialista

1 · POR QUE MODELOS DEGRADAM EM PRODUCAO

Um modelo treinado e validado com alta performance nao e eterno. O mundo muda, os dados mudam, o comportamento dos usuarios muda -- e um modelo que nao e monitorado silenciosamente perde qualidade sem que ninguem perceba, ate que o negocio sinta o impacto.

1.1 As Quatro Causas Raiz de Degradacao

Causa	Descricao	Exemplo em Telecom
Data Drift	Distribuicao das features de entrada muda ao longo do tempo	Perfil de idade muda apos nova campanha de aquisicao
Concept Drift	Relacao entre features e target muda	Padroes de churn mudam apos lancamento de novo concorrente
Data Quality Drift	Qualidade dos dados de entrada se deteriora	Pipeline ETL quebra e alimenta zeros onde deveria ter nulos
Covariate Shift	Distribuicao de X muda mas relacao X->Y permanece	Novos planos tarifarios criam perfis de uso nunca vistos no treino

1.2 O Custo do Monitoramento Ausente

- **Silencioso e gradual:** modelo continua gerando predicoes -- mas predicoes erradas. O sistema nao quebra, ele apenas deixa de ser util.
- **Decisoes de negocio erradas:** modelo de churn degradado aciona retencao de clientes que nao iam sair e ignora os que iam -- custo duplo.
- **Risco regulatorio:** modelos de credito e seguro sob LGPD/regulacao setorial precisam de explicabilidade continuada -- degradacao e passivo legal.
- **Perda de confianca:** area de negocio percebe que as recomendacoes pioraram e para de usar o modelo -- todo o investimento perdido.

1.3 O Ciclo Esperado de Degradacao

Modelos em producao tipicamente seguem um padrao: performance inicial alta, plateau de estabilidade (semanas a meses dependendo do dominio), depois queda gradual ou abrupta dependendo do tipo de drift. Em telecom, modelos de churn costumam precisar de retreinamento a cada 3 a 6 meses. Modelos de fraude podem degradar em semanas.

> **Regra pratica:** quanto mais dinamico o comportamento do negocio, mais frequente precisa ser o monitoramento e o retreinamento. Nao existe frequencia universal -- ela e definida empiricamente pelo historico de drift.

2 · TAXONOMIA DO CICLO DE VIDA DE MODELOS

Entender as fases do ciclo de vida e pre-requisito para definir o que monitorar em cada etapa.

2.1 As Fases do Ciclo de Vida

Fase	Descricao	O que Monitorar
Desenvolvimento	Treino, validacao e selecao do modelo	Metricas offline, estabilidade de CV
Staging	Validacao em ambiente similar a producao	Consistencia de scores vs treino
Deploy	Modelo em producao recebendo dados reais	Todos -- inicio do monitoramento ativo
Operacao estavel	Modelo funcionando dentro dos limites esperados	PSI, performance, data quality
Degradacao	Drift detectado, performance em queda	Alertas ativos, investigacao de causa
Retreinamento	Novo modelo treinado com dados recentes	Comparacao challenger vs champion
Aposentadoria	Modelo substituido ou descontinuado	Documentacao e handoff

2.2 Champion vs Challenger

Champion: modelo atualmente em producao, servindo predicoes reais.

Challenger: novo modelo candidato a substituir o champion. Pode rodar em paralelo recebendo uma fracao do trafego (shadow mode ou A/B) antes de ser promovido.

- **Shadow mode:** challenger recebe os mesmos inputs do champion mas suas predicoes nao sao usadas -- apenas comparadas. Zero risco de impacto em producao.
- **A/B testing:** fracao do trafego (ex: 10%) e direcionada ao challenger. Predicoes sao usadas e resultados sao comparados com o champion.
- **Criterio de promocao:** challenger supera champion em metrica principal por margem estatisticamente significativa por N dias consecutivos.

3 · DATA DRIFT -- CONCEITO, TIPOS E DETECCAO

Data drift e a mudanca na distribuicao estatistica dos dados de entrada (features X) ao longo do tempo. O modelo foi treinado numa distribuicao e esta recebendo dados de uma distribuicao diferente -- como aplicar uma receita com ingredientes diferentes.

3.1 Tipos de Data Drift

Tipo	Definicao	Como Detectar	Exemplo
Covariate Shift	P(X) muda, P(Y|X) estavel	PSI, testes estatisticos em X	Novos clientes de perfil diferente do treino
Prior Probability Shift	P(Y) muda, P(X|Y) estavel	Monitorar distribuicao do target	Taxa de churn sobe de 5% para 15%
Concept Drift	P(Y|X) muda	Monitorar performance do modelo	Padrao de fraude muda -- mesmos features, target diferente
Feature Drift	Uma ou mais features mudam distribuicao	PSI por feature, testes univariados	Codigos de renda inflacionada sem correcao
Data Quality Drift	Aumento de nulos, erros, outliers	Taxas de nulos, schema validation	Pipeline ETL com bug introduz zeros erroneos

3.2 Testes Estatisticos para Deteccao de Drift

Kolmogorov-Smirnov (KS) Test:

Testa se duas amostras vem da mesma distribuicao continua. Compara as funcoes de distribuicao acumulada (CDF) das duas amostras e calcula a maxima distancia entre elas.

H0: as duas distribuicoes sao identicas. p < 0.05 rejeita H0 -- drift detectado.

```
from scipy import stats import numpy as np # Comparar distribuicao da feature em producao vs treino def ks_drift_test(ref_data, prod_data, feature, alpha=0.05): stat, p_value = stats.ks_2samp( ref_data[feature].dropna(), prod_data[feature].dropna() ) drift = p_value < alpha return {"feature": feature, "ks_stat": stat, "p_value": p_value, "drift": drift} # Aplicar em todas as features results = [ks_drift_test(df_train, df_prod, f) for f in feature_cols] drift_df = pd.DataFrame(results).sort_values("ks_stat", ascending=False)
```

Chi-Quadrado para Categoricas:

Para features categoricas, KS nao e aplicavel. O teste qui-quadrado compara frequencias observadas vs esperadas entre as duas distribuicoes.

```
def chi2_drift_test(ref_data, prod_data, feature, alpha=0.05): # Frequencias de referencia e producao cats = set(ref_data[feature].unique()) | set(prod_data[feature].unique()) ref_counts = ref_data[feature].value_counts().reindex(cats, fill_value=1) prod_counts = prod_data[feature].value_counts().reindex(cats, fill_value=1) # Normalizar para mesma escala expected = ref_counts / ref_counts.sum() * prod_counts.sum() stat, p_value = stats.chisquare(prod_counts, f_exp=expected) return {"feature": feature, "chi2": stat, "p_value": p_value, "drift": p_value < alpha}
```

Wasserstein Distance (Earth Mover Distance):

Mede o custo minimo de "mover" uma distribuicao para que fique igual a outra. Mais intuitivo que KS para magnitude do drift -- KS diz se houve drift, Wasserstein diz o quanto.

```
from scipy.stats import wasserstein_distance dist = wasserstein_distance( df_train["arpu"].values, df_prod["arpu"].values ) print(f"Wasserstein distance: {dist:.4f}") # Interpretar em relacao ao desvio padrao da feature std_ref = df_train["arpu"].std() print(f"Distancia normalizada: {dist/std_ref:.4f} desvios padrao")
```

Jensen-Shannon Divergence:

Versao simetrica e estavel da KL Divergence. Mede a distancia entre duas distribuicoes de probabilidade. Varia entre 0 (identicas) e 1 (completamente diferentes). Aplicavel tanto para continuas (apos discretizacao) quanto para

categoricas.

```
from scipy.spatial.distance import jensenshannon import numpy as np def js_divergence(ref, prod, bins=20): # Criar histograma conjunto min_val = min(ref.min(), prod.min()) max_val = max(ref.max(), prod.max()) edges = np.linspace(min_val, max_val, bins+1) ref_hist, _ = np.histogram(ref, bins=edges, density=True) prod_hist, _ = np.histogram(prod, bins=edges, density=True) # Adicionar epsilon para evitar log(0) ref_hist = ref_hist + 1e-10 prod_hist = prod_hist + 1e-10 return jensenshannon(ref_hist, prod_hist) jsd = js_divergence(df_train["arpu"], df_prod["arpu"]) print(f"JS Divergence: {jsd:.4f}") # 0=identicas, 1=completamente diferentes
```

Teste	Tipo de Feature	O que Mede	Limitacao
KS Test	Numerica continua	Maxima distancia entre CDFs	Nao mede magnitude bem
Chi-Quadrado	Categorica	Diferenca de frequencias	Sensivel ao tamanho da amostra
Wasserstein	Numerica	Custo de transporte entre distribuicoes	Interpretacao em unidades originais
Jensen-Shannon	Qualquer (discretizada)	Distancia simetrica entre distribuicoes	Requer discretizacao para numericas
PSI	Numerica ou categorica	Estabilidade populacional	Ver Secao 4
Mann-Whitney U	Numerica	Diferenca de mediana entre grupos	Apenas localizacao, nao forma
Cramers V	Categorica	Forca da associacao entre distribuicoes	Apenas para categoricas

4 · PSI -- POPULATION STABILITY INDEX

PSI e o metodo mais amplamente adotado na industria financeira, de seguros e de telecom para monitorar estabilidade de modelos e features. Simples de calcular, intuitivo de interpretar e com thresholds de acao bem estabelecidos.

4.1 Formula e Calculo

Formula: PSI = sum[(Actual_i - Expected_i) * ln(Actual_i / Expected_i)]

Onde Expected_i e a proporcao de observacoes no bucket i durante o periodo de referencia (treino), e Actual_i e a proporcao no periodo atual (producao). O PSI e calculado sobre os buckets (faixas) da distribuicao.

Passo a passo do calculo:

1. Discretizar a feature em N buckets (tipicamente 10 ou 20 de igual frequencia na referencia).
2. Calcular proporcao de observacoes em cada bucket no periodo de referencia (Expected).
3. Calcular proporcao de observacoes em cada bucket no periodo atual (Actual).
4. Para cada bucket: calcular (Actual_i - Expected_i) * ln(Actual_i / Expected_i).
5. Somar todos os buckets -- resultado e o PSI.

Interpretacao dos thresholds:

PSI	Interpretacao	Acao Recomendada
< 0.10	Distribuicao estavel -- sem mudanca significativa	Monitorar normalmente
0.10 -- 0.20	Mudanca moderada -- investigar causas	Investigar features com maior PSI individual
> 0.20	Mudanca significativa -- modelo comprometido	Retreinar ou recalibrar modelo urgente

4.2 Implementacao Completa

```
import numpy as np import pandas as pd def calcular_psi(expected, actual, buckets=10, eps=1e-6): """ Calcula PSI entre distribuicao de referencia e atual. expected: array-like, dados de referencia (treino) actual: array-like, dados atuais (producao) buckets: numero de faixas """ # Criar breakpoints baseados nos quantis da referencia breakpoints = np.nanpercentile( expected, np.linspace(0, 100, buckets + 1) ) breakpoints = np.unique(breakpoints) # remover duplicatas # Calcular proporcoes em cada bucket def bucket_pct(data, bp): counts, _ = np.histogram(data, bins=bp) pcts = counts / len(data) return np.where(pcts == 0, eps, pcts) # evitar log(0) exp_pct = bucket_pct(expected, breakpoints) act_pct = bucket_pct(actual, breakpoints) # Formula do PSI psi_buckets = (act_pct - exp_pct) * np.log(act_pct / exp_pct) psi_total = psi_buckets.sum() # DataFrame detalhado por bucket detail = pd.DataFrame({ "bucket": range(len(exp_pct)), "expected": exp_pct, "actual": act_pct, "psi_contrib": psi_buckets }) return psi_total, detail # Aplicar em todas as features def monitorar_psi_features(df_ref, df_atual, features, buckets=10): resultados = [] for feat in features: psi, _ = calcular_psi( df_ref[feat].dropna().values, df_atual[feat].dropna().values, buckets=buckets ) status = ("ESTAVEL" if psi < 0.10 else "ATENCAO" if psi < 0.20 else "CRITICO") resultados.append({"feature": feat, "psi": psi, "status": status}) return pd.DataFrame(resultados).sort_values("psi", ascending=False)
```

4.3 PSI do Score do Modelo

Alem do PSI das features individuais, o PSI do score de saida do modelo e um dos indicadores mais importantes. Se o score muda de distribuicao, o modelo esta comportando-se diferente -- mesmo que as features pareçam estaveis.

```
# PSI do score scores_ref = model.predict_proba(X_ref)[:, 1] scores_prod = model.predict_proba(X_prod)[:, 1] psi_score, detail = calcular_psi(scores_ref, scores_prod, buckets=10) print(f"PSI do Score: {psi_score:.4f}") # Visualizar distribuicao dos scores import matplotlib.pyplot as plt plt.figure(figsize=(10,5)) plt.hist(scores_ref, bins=30, alpha=0.5, label="Referencia", density=True) plt.hist(scores_prod, bins=30, alpha=0.5, label="Producao",
```

```
density=True) plt.title(f"Distribuicao do Score -- PSI={psi_score:.4f}") plt.legend()
```

4.4 PSI para Variaveis Categoricas

Para categoricas, os buckets sao as proprias categorias -- nao e necessario discretizar.

```
def psi_categorica(ref_series, atual_series, eps=1e-6): cats = set(ref_series.unique()) |
set(atual_series.unique()) exp = ref_series.value_counts(normalize=True).reindex(cats,
fill_value=eps) act = atual_series.value_counts(normalize=True).reindex(cats, fill_value=eps)
psi_vals = (act - exp) * np.log(act / exp) return psi_vals.sum()
```

5 · CONCEPT DRIFT -- QUANDO O MUNDO MUDA

Concept drift e a mudanca na relacao entre features e target -- P(Y|X) muda. E o tipo mais perigoso de drift porque nao e detectavel apenas olhando para as features. O modelo continua recebendo dados "normais" mas o significado desses dados mudou.

5.1 Tipos de Concept Drift

Tipo	Padrao	Exemplo	Estrategia
Abrupto	Mudanca repentina em um ponto especifico	Novo concorrente lanca produto e padrao de churn muda de um dia para o outro	Detectar rapidamente e retreinar imediato
Gradual	Mudanca lenta e continua ao longo do tempo	Inflacao altera comportamento de pagamento gradualmente	Janelas deslizantes de treino (sliding window)
Incremental	Drift continuo com aceleracao progressiva	Adocao crescente de novo canal de atendimento muda perfil de reclamacoes	Monitoramento continuo + retreinamento periodico
Recorrente	Mudanca ciclica que retorna a padrao anterior	Sazonalidade anual -- churn alto em janeiro todos os anos	Modelos com componente sazonal ou retreinamento po
Blip	Mudanca temporaria que nao persiste	Evento pontual como Black Friday distorce dados por alguns dias	Identificar e ignorar periodo anomalo dos dados de tre

5.2 Como Detectar Concept Drift

Concept drift so e detectavel de forma direta quando o target esta disponivel -- o que frequentemente nao e imediato (ex: churn so e confirmado semanas apos o sinal). Ha tres abordagens principais:

Abordagem 1: Monitoramento de Performance (com labels tardios)

Quando o target fica disponivel (mesmo que com delay), calcular metricas do modelo em janelas de tempo. Queda progressiva de AUC ou KS indica concept drift.

```
# Calcular AUC por janela de tempo (mensal) from sklearn.metrics import roc_auc_score df_pred["mes"] = df_pred["data_predicao"].dt.to_period("M") auc_mensal = df_pred.groupby("mes").apply( lambda x: roc_auc_score(x["churn_real"], x["score_modelo"]) if x["churn_real"].nunique() > 1 else np.nan ).dropna() print(auc_mensal)
```

Abordagem 2: Teste de Mudanca em Erros do Modelo

Monitorar se os erros do modelo sao sistematicos -- se erros aumentam em subgrupos especificos, ha conceito mudando naquele segmento.

Abordagem 3: Proxy Drift (sem labels)

Quando o target nao esta disponivel, usar proxies: taxa de eventos de comportamento relacionados ao target (ex: aumento de ligacoes para cancelamento mesmo antes do churn formal).

5.3 Algoritmos de Deteccao de Concept Drift

Algoritmo	Mecanismo	Quando Usar
ADWIN (Adaptive Windowing)	Janela adaptativa que detecta mudancas estatisticas automaticamente	Streams de dados em tempo real -- muito eficiente
DDM (Drift Detection Method)	Monitora taxa de erro -- alarme quando supera 2 desvios	Classificadores binarios com taxa de erro conhecida
EDDM (Early Drift Detection Method)	Versao do DDM mais sensivel a drifts graduais	Quando drift gradual e esperado (mais sensivel que DDM)
Page-Hinkley Test	Detecta mudanca na media de um processo sequencial	Monitoramento de metricas agregadas ao longo do tempo
CUSUM (Cumulative Sum)	Detecta pequenas mudancas persistentes na media	Mudancas graduais -- mais sensivel que controle estatistico simple

6 · MONITORAMENTO DE PERFORMANCE

Monitorar performance e a forma mais direta de verificar se o modelo ainda cumpre seu proposito. Requer que o ground truth (target real) esteja disponivel -- o que pode ter delay em producao.

6.1 Metricas por Tipo de Problema

Problema	Metrica Principal	Metricas Complementares	Threshold de Alerta
Classificacao Binaria (Churn)	AUC-ROC	KS, Gini, F1, Precisao, Recall	Queda > 3-5pp vs baseline
Classificacao Multiclasse	Macro F1 ou Weighted F1	Accuracy, Confusion Matrix por classe	Queda > 3pp
Regressao (LTV, ARPU)	MAE ou RMSE	R2, MAPE, SMAPE	Aumento > 10-15% no MAE vs baseline
Ranking (Propensao)	KS, Gini	Lift Curve, Capture Rate por decil	KS cai > 5pp vs baseline
Deteccao de Anomalias	Precision@K	Recall, F1 em subset auditado	Precision@K cai > 10pp

6.2 Monitoramento Sem Labels Imediatos

Em muitos casos, o ground truth nao esta disponivel imediatamente. Churn e confirmado semanas apos a predicao. Fraude e auditada com delay. Nesse cenario, use proxies de performance:

- **Distribuicao do score:** se o score muda de distribuicao (PSI alto), a performance provavelmente mudou. Nao confirma, mas alerta.
- **Taxa de eventos proxy:** em churn, monitorar taxa de pedidos de cancelamento, ligacoes de reclamacao. Sao indicadores antecipados do target.
- **Calibracao sem labels:** se o modelo diz que 10% vao churnar, e o churn real foi 8%, a calibracao esta razoavel. Com delay de algumas semanas, isso e mensuravel.
- **Performance em subconjunto com labels rapidos:** se uma fracao dos eventos tem label imediato (ex: cancelamento voluntario vs involuntario), monitorar performance nesse subconjunto como proxy.

6.3 Metricas por Decil -- Lift e Capture Rate

Em modelos de churn e propensao, monitorar o lift por decil e mais informativo que a AUC sozinha. Um modelo pode manter AUC mas perder poder de discriminacao nos decis mais altos -- exatamente onde as acoes de negocio sao tomadas.

```
# Calcular lift por decil def lift_table(y_true, y_score, n_decis=10): df = pd.DataFrame({"target": y_true, "score": y_score}) df["decil"] = pd.qcut(df["score"], q=n_decis, labels=False, duplicates="drop") df["decil"] = df["decil"].max() - df["decil"] # inverter: decil 1 = maior score base_rate = df["target"].mean() lift = df.groupby("decil")["target"].agg(["mean","count"]) lift["lift"] = lift["mean"] / base_rate lift["capture_rate"] = ( df.groupby("decil")["target"].sum() / df["target"].sum() ) return lift.reset_index()
```

7 · MONITORAMENTO DE FEATURES E DADOS

Monitorar os dados de entrada e a primeira linha de defesa. Problemas detectados aqui podem ser corrigidos antes de impactar o modelo.

7.1 Data Quality Monitoring

Dimensao	O que Monitorar	Metrica	Threshold Tipico
Completude	Taxa de nulos por feature	% de nulos	Desvio > 2x da media historica
Unicidade	Duplicatas no dataset	% de linhas duplicadas	Qualquer valor > 0 para IDs
Validade de schema	Tipos de dado corretos	Erros de tipo	Zero tolerancia
Range	Valores dentro do dominio esperado	% fora do range	> 0.1% fora do P0.1-P99.9
Freshness	Latencia dos dados de entrada	Horas desde ultimo dado	Definir SLA por pipeline
Volume	Numero de registros processados	Contagem diaria	Desvio > 20% da media
Consistencia	Features derivadas consistentes com bases	Reconciliacao cruzada	Zero tolerancia

7.2 Implementacao de Data Quality Checks

```
import pandas as pd import numpy as np def data_quality_report(df, df_ref=None): """ Gera relatorio completo de qualidade dos dados. df_ref: dataset de referencia para comparacoes historicas """ report = {} for col in df.columns: null_pct = df[col].isnull().mean() nunique = df[col].nunique() dtype = str(df[col].dtype) stats = { "null_pct": null_pct, "nunique": nunique, "dtype": dtype } if df[col].dtype in [np.float64, np.int64]: stats.update({ "mean": df[col].mean(), "std": df[col].std(), "min": df[col].min(), "max": df[col].max(), "p01": df[col].quantile(0.01), "p99": df[col].quantile(0.99) }) # Comparar com referencia se disponivel if df_ref is not None and col in df_ref.columns: psi, _ = calcular_psi( df_ref[col].dropna().values, df[col].dropna().values ) stats["psi"] = psi stats["psi_status"] = ( "ESTAVEL" if psi < 0.10 else "ATENCAO" if psi < 0.20 else "CRITICO" ) report[col] = stats return pd.DataFrame(report).T
```

7.3 Schema Validation com Great Expectations

Great Expectations e a biblioteca padrao do mercado para validacao de qualidade de dados em pipelines de ML. Define "expectativas" sobre os dados e verifica automaticamente a cada execucao.

```
import great_expectations as ge # Criar suite de expectativas df_ge = ge.from_pandas(df_producao) # Definir expectativas df_ge.expect_column_values_to_not_be_null("arpu") df_ge.expect_column_values_to_be_between("arpu", min_value=0, max_value=10000) df_ge.expect_column_values_to_be_in_set("tipo_plano", ["basico","premium","controle"]) df_ge.expect_column_values_to_be_of_type("churn", "int64") df_ge.expect_table_row_count_to_be_between(min_value=1000, max_value=1000000) # Rodar validacao results = df_ge.validate() print(f"Sucesso: {results.success}") print(f"Falhas: {results.statistics['unsuccessful_expectations']}")
```

8 · MONITORAMENTO DE PREDICTIONS (OUTPUT DRIFT)

Monitorar a distribuicao das predicoes e o sinal mais direto de que algo mudou no comportamento do modelo. Mesmo sem ter o ground truth, se o modelo comecou a gerar scores muito diferentes do historico, algo aconteceu.

8.1 O que Monitorar nos Outputs

• **Distribuicao do score:** PSI entre distribuicao atual e de referencia. Threshold: PSI > 0.10 investigar, > 0.20 alarme.

• **Taxa de eventos preditos:** se o modelo prediz 5% de churn historicamente e hoje esta predizendo 15%, algo mudou significativamente.

• **Distribuicao por decil:** proporcao de clientes em cada decil de score. Se concentracao nos extremos muda, os cortes de acao sao afetados.

• **Score medio por segmento:** se o score medio do segmento premium mudou mas do basico nao, o drift e segmentado.

• **Volatilidade de score:** o mesmo cliente deveria ter score similar em dias consecutivos. Alta variabilidade pode indicar instabilidade do modelo.

8.2 Monitoramento de Score ao Longo do Tempo

```
# Monitoramento diario do score def monitorar_score_diario(df_scores, df_ref_scores, coluna_score="score", coluna_data="data"): resultados = [] ref_scores = df_ref_scores[coluna_score].values for data, grupo in df_scores.groupby(coluna_data): prod_scores = grupo[coluna_score].values psi, _ = calcular_psi(ref_scores, prod_scores) ks_stat, ks_p = stats.ks_2samp(ref_scores, prod_scores) mean_score = prod_scores.mean() pct_alto = (prod_scores > 0.5).mean() # % acima do corte resultados.append({ "data": data, "psi": psi, "ks_stat": ks_stat, "ks_pvalue": ks_p, "mean_score": mean_score, "pct_score_alto": pct_alto, "n_predicoes": len(prod_scores), "status": ("CRITICO" if psi > 0.20 else "ATENCAO" if psi > 0.10 else "ESTAVEL") }) return pd.DataFrame(resultados)
```

9 · SHAP EM MONITORAMENTO

SHAP values em producao sao muito mais do que uma ferramenta de explicacao -- sao um radar de monitoramento sofisticado. Mudancas nos SHAP values revelam nao apenas QUE o modelo mudou, mas ONDE e POR QUE.

9.1 SHAP Mean Drift

Monitorar a media dos SHAP values de cada feature ao longo do tempo. Se o SHAP medio de uma feature muda significativamente, essa feature esta tendo influencia diferente no modelo.

```
# Calcular SHAP medio por feature por periodo def shap_drift_report(explainer, X_ref, X_prod, feature_names): shap_ref = np.abs(explainer.shap_values(X_ref)).mean(axis=0) shap_prod = np.abs(explainer.shap_values(X_prod)).mean(axis=0) report = pd.DataFrame({ "feature": feature_names, "shap_ref": shap_ref, "shap_prod": shap_prod, "variacao_abs": shap_prod - shap_ref, "variacao_pct": (shap_prod - shap_ref) / (shap_ref + 1e-10) * 100 }).sort_values("variacao_abs", key=abs, ascending=False) return report
```

9.2 SHAP Rank Stability

Uma forma robusta de detectar concept drift: verificar se o ranking de importancia das features (por SHAP medio) mudou entre dois periodos. Mudanca de ranking indica que o modelo esta aprendendo relacoes diferentes.

```
# Correlacao de Spearman entre rankings de SHAP from scipy.stats import spearmanr rank_ref = np.abs(shap_values_ref).mean(axis=0).argsort()[::-1] rank_prod = np.abs(shap_values_prod).mean(axis=0).argsort()[::-1] corr, p = spearmanr(rank_ref, rank_prod) print(f"Correlacao de ranking SHAP: {corr:.4f}") # Corr > 0.9: rankings estaveis # Corr < 0.7: mudanca significativa na logica do modelo
```

9.3 Top Feature Tracking

Monitorar qual e a feature mais importante do modelo ao longo do tempo. Se a feature #1 muda de identidade, o modelo aprendeu uma logica completamente diferente.

```
# Historico mensal da top feature def top_feature_historico(df_scores_mes, explainer, feature_names): historico = [] for mes, grupo in df_scores_mes.groupby("mes"): X_mes = grupo[feature_names] sv = np.abs(explainer.shap_values(X_mes)).mean(axis=0) top_idx = sv.argmax() historico.append({ "mes": mes, "top_feature": feature_names[top_idx], "shap_top": sv[top_idx], "shap_values": dict(zip(feature_names, sv)) }) return pd.DataFrame(historico)
```

10 · ALERTAS, SLAs E ESTRATEGIAS DE RESPOSTA

Monitorar sem um sistema de alertas e como ter um alarme de incendio mas nunca olhar para ele. Alertas bem definidos transformam monitoramento em acao.

10.1 Hierarquia de Alertas

Nivel	Condicao	Acao	Urgencia
INFO	PSI entre 0.05-0.10 em qualquer feature	Registrar no log, incluir no relatorio semanal	Nenhuma acao imediata
WARNING	PSI entre 0.10-0.20 em feature critica OU queda de 2-3 pp em AUC	Notificar time de DS, investigar causa raiz	Investigar em 48h
CRITICAL	PSI > 0.20 em feature critica OU score medio deslocou > 5 pp em relacao a baseline	Alerta imediato para o time de DS + product owner	Avaliar retreinamento emergencial em 24h
INCIDENT	Falha de pipeline, schema quebrado, volume zero, modelo retornando constante	Acionar on-call de engenharia + DS	Acao imediata, postmortem

10.2 SLAs de Monitoramento

Metrica	Frequencia de Calculo	Frequencia de Revisao Humana	SLA de Resposta
Data quality (nulos, schema)	A cada batch/hora	Diaria automatizada	2h para CRITICAL
PSI das features	Diario	Semanal com revisao humana	48h para WARNING
PSI do score	Diario	Diaria automatizada	24h para CRITICAL
Performance (AUC/KS)	Semanal ou com labels disponiveis	Mensal	48h para WARNING
SHAP drift	Mensal	Mensal	72h para mudanca significativa
Volume de predicoes	Horario	Diaria	1h para anomalia
Latencia de predicao	Continuo	Diaria	30min para degradacao

10.3 Runbook de Resposta a Alertas

Quando PSI > 0.20 numa feature critica:

1. Identificar qual feature tem PSI alto e em qual periodo iniciou.
2. Verificar se houve mudanca no pipeline de dados (schema, query, transformacao).
3. Verificar se houve evento de negocio que explique a mudanca (campanha, produto novo).
4. Calcular PSI do score -- se tambem alto, modelo esta afetado.
5. Calcular performance no periodo mais recente com labels disponiveis.
6. Decidir: retreinar com dados recentes ou aguardar mais dados.
7. Documentar causa raiz e acao tomada.

11 · RETREINAMENTO -- QUANDO E COMO

Retreinar nao e simplesmente rodar o mesmo codigo com dados novos. E um processo estruturado que inclui decisao, execucao, validacao e deploy controlado.

11.1 Estrategias de Janela de Treino

Estrategia	Descricao	Vantagem	Desvantagem
Full Retrain	Treinar com todos os dados historicos disponiveis	Maximiza volume de dados	Dados antigos podem prejudicar se ha drift
Sliding Window	Treinar apenas com os N meses mais recentes	Adapta-se a drifts recentes, descarta padroes obsoletos	Perde padroes de longo prazo, instavel com janelas pequ
Expanding Window	Adicionar dados novos mantendo os antigos	Cresce com o tempo, mantem historico	Dados antigos com peso igual a recentes -- pode ser preju
Weighted Window	Treinar com dados historicos mas com peso decrescente para dados mais antigos	Equilibra estabilidade e adaptacao	Requer definicao da funcao de decaimento a ser calibrado
Incremental/Online	Atualizar modelo incrementalmente com novos dados	Adapta-se continuamente sem retreinar do zero	Implementacao complexa, risco de catastrophic forgetting

11.2 Criterios de Decisao de Retreinamento

- **Trigger por PSI:** PSI > 0.20 em features criticas ou no score.
- **Trigger por performance:** AUC ou KS caiu > 5pp vs baseline.
- **Trigger por tempo:** retreinamento preventivo periodico mesmo sem drift detectado (mensal, trimestral, semestral -- definido empiricamente).
- **Trigger por evento de negocio:** lancamento de produto, mudanca de portfólio, aquisicao de empresa, mudanca regulatoria.
- **Trigger por volume:** acumulou X% mais dados que o treino original -- vale retreinar para capturar novos padroes.

11.3 Validacao Antes do Deploy do Novo Modelo

Regra de ouro: novo modelo SEM validacao nao vai para producao.

- **Performance offline:** novo modelo deve superar champion no conjunto de validacao. Margem minima: 1-2pp em AUC para justificar mudanca.
- **Estabilidade:** testar novo modelo em multiplas janelas de tempo, nao apenas no periodo mais recente.
- **Consistencia de scores:** clientes de alto risco no champion devem ter scores altos no challenger (correlacao Spearman > 0.85 entre scores dos dois modelos).
- **Teste de segmento:** validar performance em segmentos criticos separadamente (premium vs basico, por regiao, por safra).
- **Shadow mode:** rodar challenger em producao sem usar suas predicoes por N dias antes de promover.

11.4 Codigo de Comparacao Champion vs Challenger

```
from sklearn.metrics import roc_auc_score from scipy import stats def comparar_modelos(champion, challenger, X_val, y_val, feature_names, n_bootstrap=1000): """ Compara champion vs challenger com bootstrap para significancia estatistica. """ auc_champ = roc_auc_score(y_val, champion.predict_proba(X_val)[:,1]) auc_chall = roc_auc_score(y_val, challenger.predict_proba(X_val)[:,1]) # Bootstrap para intervalo de confianca diffs = [] n = len(y_val) for _ in range(n_bootstrap): idx = np.random.choice(n, n, replace=True) auc_c = roc_auc_score(y_val[idx], champion.predict_proba(X_val[idx])[:,1]) auc_h = roc_auc_score(y_val[idx], challenger.predict_proba(X_val[idx])[:,1]) diffs.append(auc_h - auc_c) ci_low, ci_high = np.percentile(diffs, [2.5, 97.5]) significativo = ci_low > 0 # IC nao inclui zero print(f"AUC Champion: {auc_champ:.4f}") print(f"AUC Challenger: {auc_chall:.4f}") print(f"Diferenca: {auc_chall - auc_champ:+.4f}") print(f"IC 95%: [{ci_low:.4f}, {ci_high:.4f}]") print(f"Significativo:
```

```
{significativo}") return {"promover": significativo and auc_chall > auc_champ, "auc_diff": auc_chall
- auc_champ, "ci": (ci_low, ci_high)}
```

12 · MLOPS -- FERRAMENTAS E ARQUITETURA

MLOps e a disciplina que une desenvolvimento de ML e operacoes de software. Monitoramento e apenas uma das suas dimensoes -- mas e a mais critica para sustentabilidade de modelos em producao.

12.1 As Quatro Dimensoes do MLOps

Dimensao	O que Cobre	Ferramentas Principais
Model Registry	Versionamento e catalogo de modelos treinados	MLflow, Weights & Biases, Vertex AI Model Registry
Feature Store	Armazenamento centralizado de features para treino e inferencia	Feast, Tecton, Vertex AI Feature Store, Databricks Feature Store
Pipeline de CI/CD	Automacao de retreinamento, validacao e deploy	MLflow, Kubeflow, Vertex AI Pipelines, Github Actions
Monitoramento	Tracking de performance, drift e data quality em producao	Evidently AI, Whylogs, Arize, Fiddler, Grafana + custom

12.2 Evidently AI -- Biblioteca de Monitoramento

Evidently e a biblioteca open source mais completa para monitoramento de ML em Python. Gera relatorios de drift, qualidade de dados e performance com uma linha de codigo.

```
from evidently.report import Report from evidently.metric_preset import ( DataDriftPreset, DataQualityPreset, TargetDriftPreset, ClassificationPreset ) from evidently.metrics import * # Relatorio completo de drift report = Report(metrics=[ DataDriftPreset(), # PSI + testes de drift por feature DataQualityPreset(), # nulos, outliers, qualidade geral TargetDriftPreset(), # drift no target ]) report.run( reference_data=df_referencia, # dados de treino current_data=df_producao # dados atuais ) # Exportar relatorio report.save_html("relatorio_drift.html") # Verificar se ha drift result = report.as_dict() drift_detected = result["metrics"][0]["result"]["dataset_drift"]
```

12.3 MLflow para Model Registry e Tracking

```
import mlflow import mlflow.xgboost # Registrar treino e metricas with mlflow.start_run(run_name="churn_model_v3"): # Treinar modelo model.fit(X_train, y_train) # Log de metricas mlflow.log_metric("auc_treino", auc_train) mlflow.log_metric("auc_val", auc_val) mlflow.log_metric("ks_val", ks_val) mlflow.log_metric("psi_score", psi_score) # Log de parametros mlflow.log_params(model.get_params()) # Registrar modelo mlflow.xgboost.log_model(model, "model") # Registrar no Model Registry mlflow.register_model( f"runs:/{mlflow.active_run().info.run_id}/model", "churn_model" ) # Promover para staging client = mlflow.MlflowClient() client.transition_model_version_stage( name="churn_model", version=3, stage="Staging" )
```

12.4 Arquitetura de Monitoramento em Producao

Componente	Responsabilidade	Tecnologia Sugerida
Coleta de dados	Capturar features de entrada e scores de saida em producao	Kafka, Pub/Sub, logs estruturados
Armazenamento	Guardar historico de features, scores e labels	BigQuery, Snowflake, Delta Lake
Calculo de metricas	PSI, KS, AUC, qualidade de dados -- agendado	Airflow, Cloud Scheduler, DBT
Dashboard	Visualizacao de metricas ao longo do tempo	Grafana, Looker, Power BI, Superset
Alertas	Notificacao quando thresholds sao ultrapassados	PagerDuty, Slack webhook, email
Model Registry	Versionar e gerenciar modelos e suas metricas	MLflow, Vertex AI, SageMaker
Re-training pipeline	Automatizar retreinamento quando drift e detectado	Kubeflow, Vertex AI Pipelines, Airflow

13 · PIPELINE COMPLETO DE MONITORAMENTO EM PRODUCAO

13.1 Codigo do Pipeline Integrado

```
import pandas as pd import numpy as np from scipy import stats from sklearn.metrics import roc_auc_score import warnings class ModelMonitor: """ Pipeline completo de monitoramento para modelos de classificacao. Cobre: PSI, drift de features, performance, data quality e score drift. """ def __init__(self, model, explainer, X_ref, y_ref, feature_names, psi_warn=0.10, psi_crit=0.20, auc_drop_warn=0.03, auc_drop_crit=0.05): self.model = model self.explainer = explainer self.X_ref = X_ref self.y_ref = y_ref self.feature_names = feature_names self.psi_warn = psi_warn self.psi_crit = psi_crit self.auc_baseline = roc_auc_score( y_ref, model.predict_proba(X_ref)[:,1]) self.scores_ref = model.predict_proba(X_ref)[:,1] self.auc_drop_warn = auc_drop_warn self.auc_drop_crit = auc_drop_crit def run(self, X_prod, y_prod=None, periodo="atual"): """ Executa monitoramento completo para um periodo de producao. """ relatorio = {"periodo": periodo, "alertas": [], "metricas": {}} # 1. DATA QUALITY null_pcts = X_prod.isnull().mean() for feat, null_pct in null_pcts.items(): ref_null = self.X_ref[feat].isnull().mean() if null_pct > ref_null * 2 + 0.05: relatorio["alertas"].append( f"WARNING: nulos em {feat}: {null_pct:.1%} vs ref {ref_null:.1%}") # 2. PSI POR FEATURE psi_features = {} for feat in self.feature_names: psi_val, _ = calcular_psi( self.X_ref[feat].dropna().values, X_prod[feat].dropna().values ) psi_features[feat] = psi_val if psi_val > self.psi_crit: relatorio["alertas"].append( f"CRITICAL: PSI de {feat} = {psi_val:.4f}") elif psi_val > self.psi_warn: relatorio["alertas"].append( f"WARNING: PSI de {feat} = {psi_val:.4f}") relatorio["metricas"]["psi_features"] = psi_features # 3. PSI DO SCORE scores_prod = self.model.predict_proba(X_prod.fillna(0))[:,1] psi_score, _ = calcular_psi(self.scores_ref, scores_prod) relatorio["metricas"]["psi_score"] = psi_score if psi_score > self.psi_crit: relatorio["alertas"].append( f"CRITICAL: PSI do Score = {psi_score:.4f}") # 4. PERFORMANCE (se labels disponiveis) if y_prod is not None: auc_prod = roc_auc_score(y_prod, scores_prod) auc_drop = self.auc_baseline - auc_prod relatorio["metricas"]["auc_producao"] = auc_prod relatorio["metricas"]["auc_drop"] = auc_drop if auc_drop > self.auc_drop_crit: relatorio["alertas"].append( f"CRITICAL: AUC caiu {auc_drop:.4f} " f"({self.auc_baseline:.4f} -> {auc_prod:.4f})") elif auc_drop > self.auc_drop_warn: relatorio["alertas"].append( f"WARNING: AUC caiu {auc_drop:.4f}") # 5. STATUS GERAL if any("CRITICAL" in a for a in relatorio["alertas"]): relatorio["status"] = "CRITICAL" elif any("WARNING" in a for a in relatorio["alertas"]): relatorio["status"] = "WARNING" else: relatorio["status"] = "ESTAVEL" return relatorio
```

13.2 Fluxo de Decisao de Retreinamento

Condicao	Decisao	Acao
Sem drift, performance estavel	Manter champion	Continuar monitoramento
PSI moderado (0.10-0.20), performance OK	Investigar	Analise de causa raiz, aguardar
PSI alto (>0.20), performance OK	Retreinar preventivo	Challenger com dados recentes
PSI moderado, performance caindo	Retreinar urgente	Sliding window ou expanding window
PSI alto, performance caindo	Retreinar emergencial	Rollback + retreinamento imediato
Schema quebrado / pipeline com erro	Incidente	Rollback + correcao do pipeline
Challenger supera champion por 5+ dias	Promover challenger	Deploy controlado com A/B

14 · CHECKLIST DO ESPECIALISTA

Use para auditar o monitoramento de qualquer modelo em producao. Item nao marcado = risco operacional real.

	Item	Area
■	Baseline de performance documentado no deploy (AUC, KS, MAE)?	Deploy
■	Distribuicao de referencia de todas as features salvas?	Deploy
■	Distribuicao de referencia do score salva?	Deploy
■	Pipeline de coleta de features e scores em producao configurado?	Infraestrutura
■	PSI calculado diariamente para features criticas?	Drift
■	PSI calculado diariamente para o score de saida?	Drift
■	Teste KS ou Chi2 configurado para deteccao de drift?	Drift
■	Monitoramento de nulos por feature configurado?	Data Quality
■	Monitoramento de schema (tipos, colunas) configurado?	Data Quality
■	Monitoramento de volume (contagem de registros) configurado?	Data Quality
■	Monitoramento de performance com labels configurado?	Performance
■	Proxies de performance definidos para periodos sem labels?	Performance
■	SHAP drift monitorado mensalmente?	Interpretabilidade
■	Thresholds de alerta definidos para cada metrica?	Alertas
■	SLA de resposta definido para cada nivel de alerta?	Alertas
■	Runbook de investigacao de drift documentado?	Processo
■	Criterio de decisao de retreinamento documentado?	Retreinamento
■	Estrategia de janela de treino definida?	Retreinamento
■	Processo de validacao do challenger documentado?	Retreinamento
■	Model registry com versoes e metricas de cada modelo?	MLOps
■	Dashboard de monitoramento acessivel para o time?	MLOps
■	Processo de rollback documentado e testado?	Contingencia
■	Postmortem de incidentes de drift documentados?	Processo
■	Frequencia de revisao humana dos relatorios definida?	Processo